

CS 156

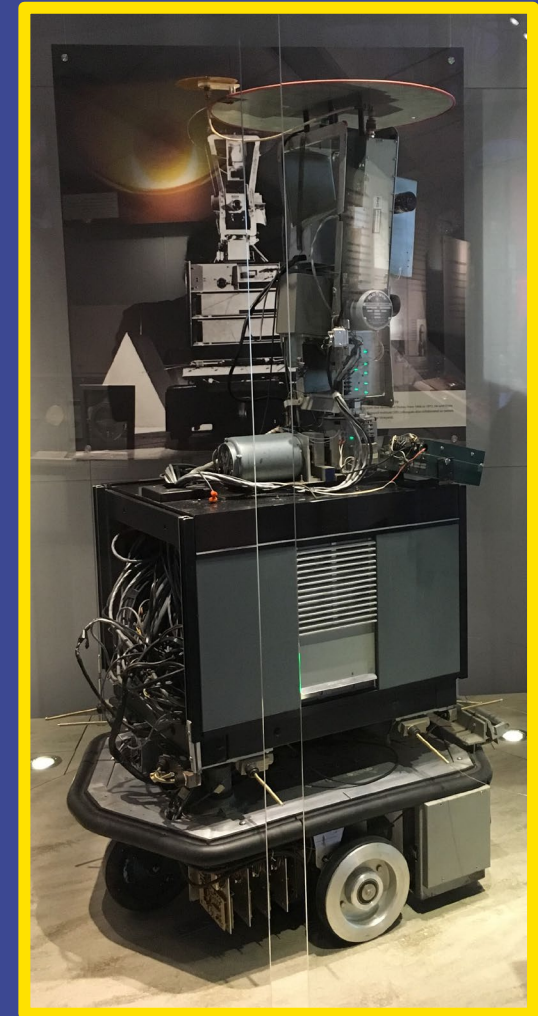
Intro to AI

Classical Planning

Shakey

Developed at SRI International 1966-1972

Retired at the Computer History Museum



Today

- ▶ Classical Planning
 - Representing a planning problem
 - Solving a planning problem
 - Generating Heuristics

Planning

- ▶ Planning: coming up with a plan of actions to achieve some goals
- ▶ Planning is a central part of AI
- ▶ Search based planning agents:
 - work in deterministic, static and fully observable environments
 - atomic representations of states
 - need domain specific knowledge (heuristic)
- ▶ Classical Planning
 - work in deterministic, static and fully observable environments
 - factored representation of states
 - Combine logic and search

The Frame Problem

Let's take as an example this classroom. To represent the current state, we can say that:

- ▶ The first window is closed
- ▶ The door is open
- ▶ The lights are on

Now let me open the window. What are the effects of my action?

- ? The first window is open
- ? The door is open
- ? The lights are on

The Frame Problem

Do we need to specify the result of an action on ALL the state variables?

The Frame Problem

The frame problem is the challenge of representing the effects of an action without having to represent explicitly a large number of intuitively obvious non-effects.

How do we capture what is relevant and what is not relevant?

The Frame Problem

Common sense law of inertia:

- ▶ an action can be assumed not to change a variable unless there is evidence to the contrary.

From STRIPS to PDDL

STRIPS (**S**tanford **R**esearch **I**nstitute **P**roblem **S**olver) was initially an automated planner developed for Shakey at SRI in 1971.

The same name was later used to refer to the **formal language used for the inputs to this planner**.

This language is the base for most of the languages used in planning today.

PDDL - Planning Domain Definition Language, developed in 1998 is derived from the STRIPS planning language.

PDDL

PDDL includes:

- ▶ Initial and goal **states**.
- ▶ A set of **action schemas** in terms of **preconditions** and **effects**.

PDDL – State Representation

A state is represented with a **conjunction** of FOL literals representing atomic facts (fluents):

- Example: $\text{At}(\text{Sam}, \text{SJSU}) \wedge \text{At}(\text{Plane1}, \text{SFO}) \wedge \text{At}(\text{Plane2}, \text{SJC})$

The literals must be **ground** (no variables):

- $\text{At}(x, \text{SFO})$ is not allowed

The literals must be **function-free**:

- $\text{At}(\text{Brother}(\text{Anna}), \text{SJSU})$ is not allowed

Names are unique:

- $\text{At}(\text{Plane1}, \text{SFO}) \wedge \text{At}(\text{Plane2}, \text{SJC})$: Plane1 and Plane2 are **distinct**

PDDL – State Representation

Closed World Assumption:

- Whatever is not explicitly stated is assumed to be false.
- There is no need for negative literals in a state representation.

PDDL – State Representation

Which of the following is the most concise and sufficient representation of the state where only plane P1 is at SFO, P2 is also a plane but it is not at SFO?

- A. $\text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{At}(P1, \text{SFO}) \wedge \neg \text{At}(P2, \text{SFO}) \wedge \text{Airport}(\text{SFO})$
- B. $\text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{At}(P1, \text{SFO}) \wedge \text{Airport}(\text{SFO})$
- C. $\text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{At}(P1, \text{SFO}) \wedge \neg \text{At}(P2, \text{SFO}) \wedge \text{Airport}(\text{SFO}) \wedge \neg \text{Airport}(P1) \wedge \neg \text{Airport}(P2) \wedge \neg \text{Plane}(\text{SFO})$
- D. $\text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{At}(P1, \text{SFO}) \wedge \neg \text{At}(P2, \text{SFO}) \wedge \text{Airport}(\text{SFO}) \wedge \neg \text{Airport}(P1) \wedge \neg \text{Airport}(P2) \wedge \neg \text{Plane}(\text{SFO}) \wedge P1 \neq P2$

PDDL – State Representation

Is the following a valid state representation in PDDL?

$\text{At}(\text{Anna}, \text{SJSU}) \wedge \text{At}(x, \text{SJSU}) \wedge \text{Knows}(\text{Anna}, x)$

- A. Yes
- B. No

PDDL – State Representation

Is the following a valid state representation in PDDL?

$\text{At}(\text{Zoe}, \text{SJSU}) \wedge \text{At}(\text{Mother}(\text{Zoe}), \text{SJSU})$

A. Yes

B. No

PDDL – State Representation

Initial state:

Init(At(P1, SFO) $\wedge$ At(P2, JFK) $\wedge$ Plane(P1) $\wedge$ Plane(P2) $\wedge$ Airport(SFO) $\wedge$
Airport(JFK))

PDDL – Action Representation

Actions (STRIPS operators) are described using **action schemas**.

An action schema includes:

- **Action name** followed by a list of **variables**
- Precondition: conjunction of positive function-free literals
- Effect: conjunction of positive or negative function-free literals

Example:

Action(**Fly**(**p**, **from**, **to**),

PRECOND: $\text{At}(\text{p}, \text{from}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(\text{p}, \text{from}) \wedge \text{At}(\text{p}, \text{to})$

PDDL – Action Representation

Action(**Fly**(*p*, *from*, *to*),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

Each schema represents **a set of actions**.

We can think of the variables *p*, *from* and *to* as **universally quantified**.

We can substitute values for the variables and get one **ground action**:
{p/Plane1, from/SFO, to/JFK}:

Action(**Fly**(**Plane1**, **SFO**, **JFK**),

PRECOND: $\text{At}(\text{Plane1}, \text{SFO}) \wedge \text{Plane}(\text{Plane1}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$

EFFECT: $\neg \text{At}(\text{Plane1}, \text{SFO}) \wedge \text{At}(\text{Plane1}, \text{JFK})$)

PDDL – Action Representation

Action(Fly(p, from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

- ▶ Any variable in the effect of an action schema must appear in the action's preconditions.
- ▶ There is no explicit reference to time: the precondition implicitly refers to time t and the effect to time $t+1$.

PDDL – Action Representation

Is the following a valid action schema?

Action(Go(x, from, to),

PRECOND: $\text{At}(x, \text{from})$

EFFECT: $\neg \text{At}(x, \text{from}) \wedge \text{At}(x, \text{to})$)

A. Yes

B. No

PDDL and the Frame Problem

Classical planning deals with problems where most actions leave most things unchanged.

The frame problem is the challenge of representing the effects of an action without having to explicitly represent non-effects.

PDDL addresses the frame problem by specifying the result of an action in terms of what changes. Anything that stays the same is not mentioned.

PDDL – Action Representation

Action(Drive(x, car, from, to)

PRECOND: $\text{At}(x, \text{from}) \wedge \text{At}(\text{car}, \text{from}) \wedge \text{HasDriversLicense}(x)$
 $\wedge \text{Location}(\text{from}) \wedge \text{Location}(\text{to})$

EFFECT:?

- A. EFFECT: $\text{At}(x, \text{to}) \wedge \neg \text{At}(x, \text{from})$
- B. EFFECT: $\neg \text{At}(x, \text{from}) \wedge \neg \text{At}(\text{car}, \text{from}) \wedge \text{HasDriversLicense}(x)$
- C. EFFECT: $\text{At}(x, \text{to}) \wedge \text{At}(\text{car}, \text{to}) \wedge \text{HasDriversLicense}(x)$
- D. EFFECT: $\text{At}(x, \text{to}) \wedge \text{At}(\text{car}, \text{to}) \wedge \neg \text{At}(x, \text{from}) \wedge \neg \text{At}(\text{car}, \text{from})$
- E. EFFECT: $\text{At}(x, \text{to}) \wedge \text{At}(\text{car}, \text{to}) \wedge \neg \text{At}(x, \text{from}) \wedge \neg \text{At}(\text{car}, \text{from}) \wedge \text{HasDriversLicense}(x)$

PDDL – Applicable Actions

An action a is **applicable** in state s if a 's preconditions are satisfied by s .
When an action schema contains variables, it may have **multiple applicable instantiations**.

PDDL – Applicable Actions

Init($\text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P2}, \text{JFK}) \wedge \text{Plane}(\text{P1}) \wedge \text{Plane}(\text{P2}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$)

Action(Fly(p , from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

The Fly action can be instantiated by unifying a state s with the preconditions:
 $\{p/\text{P1}, \text{from}/\text{SFO}, \text{to}/\text{JFK}\}$

Action(Fly(P1 , SFO, JFK),

PRECOND: $\text{At}(\text{P1}, \text{SFO}) \wedge \text{Plane}(\text{P1}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$

EFFECT: $\neg \text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P1}, \text{JFK})$)

PDDL – Applicable Actions

Init($\text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P2}, \text{JFK}) \wedge \text{Plane}(\text{P1}) \wedge \text{Plane}(\text{P2}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$)

Action($\text{Fly}(\text{p}, \text{from}, \text{to})$,

PRECOND: $\text{At}(\text{p}, \text{from}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(\text{p}, \text{from}) \wedge \text{At}(\text{p}, \text{to})$)

The Fly action can also be instantiated as:

Action($\text{Fly}(\text{P2}, \text{JFK}, \text{SFO})$,

PRECOND: $\text{At}(\text{P2}, \text{JFK}) \wedge \text{Plane}(\text{P2}) \wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO})$

EFFECT: $\neg \text{At}(\text{P2}, \text{JFK}) \wedge \text{At}(\text{P2}, \text{SFO})$)

Both ground actions are applicable in the initial state.

PDDL – Applying the Action

Starting from a state s , to apply action a :

- ▶ Remove the **negative literals in the action's effect**. This is the delete list $\text{Del}(a)$
- ▶ Add the **positive literals** in the action's effects. This is the add list or $\text{Add}(a)$
- ▶ $\text{Result}(s,a) = (s - \text{Del}(a)) \cup \text{Add}(a)$

$s: \text{At}(P1, \text{SFO}) \wedge \text{At}(P2, \text{JFK}) \wedge \text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$

$a: \text{Action}(\text{Fly}(P1, \text{SFO}, \text{JFK}),$

$\text{PRECOND}: \text{At}(P1, \text{SFO}) \wedge \text{Plane}(P1) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$

$\text{EFFECT}: \neg \text{At}(P1, \text{SFO}) \wedge \text{At}(P1, \text{JFK})$)

Delete list $\text{Del}(a): \text{At}(P1, \text{SFO})$

Add list $\text{Add}(a): \text{At}(P1, \text{JFK})$

$\text{Result}(s,a): \text{At}(P2, \text{JFK}) \wedge \text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK}) \wedge \text{At}(P1, \text{JFK})$

PDDL – Goal Representation

A goal is a conjunction of literals, positive or negative that may contain variables.

Variables are treated as **existentially quantified**.

A state is a goal state if it entails the goal.

$\text{Goal}(\text{At}(x, \text{SFO}) \wedge \text{Plane}(x))$

Our goal here is to have at least one plane at SFO.

s:

$\text{At}(\text{Plane1}, \text{SFO}) \wedge \text{Plane}(\text{Plane1}) \wedge \text{At}(\text{Plane2}, \text{JFK}) \wedge \text{Plane}(\text{Plane2})$

The state s entails the goal. It is a goal state.

PDDL – Problem Representation

A set of action schemas defines a **planning domain**.

A specific problem within the domain is defined with the addition of an initial state and a goal.

The problem is solved when we find **a sequence of actions** from the initial state to a state that entails the goal.

PDDL – Cargo Example

Planning Domain:

Actions

Load
Unload
Fly

Predicates

In(c, p)
At(x, a)

- ▶ When a plane flies from one airport to another all the cargo inside the plane goes with it.
- ▶ A piece of cargo ceases to be 'At' anywhere when it is in a plane.
- ▶ The cargo becomes 'At' the new airport when it is unloaded.

PDDL – Cargo Example

Planning Domain:

Action(Load(c, p, a),

PRECOND: $\text{At}(c, a) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\neg \text{At}(c, a) \wedge \text{In}(c, p)$)

Action(Unload(c, p, a),

PRECOND: $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\text{At}(c, a) \wedge \neg \text{In}(c, p)$)

Action(Fly(p, from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

PDDL – Cargo Example

A specific problem = initial state + goal + planning domain

Init($\text{At}(\text{C1}, \text{SFO}) \wedge \text{At}(\text{C2}, \text{JFK}) \wedge \text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P2}, \text{JFK}) \wedge \text{Cargo}(\text{C1}) \wedge$
 $\text{Cargo}(\text{C2}) \wedge \text{Plane}(\text{P1}) \wedge \text{Plane}(\text{P2}) \wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO}))$

Goal($\text{At}(\text{C1}, \text{JFK}) \wedge \text{At}(\text{C2}, \text{SFO})$)

Action(Load(c, p, a),
...)

Action(Unload(c, p, a),
...)

Action(Fly(p, from, to),
...)

PDDL – Cargo Example

Init($\text{At}(\text{C1}, \text{SFO}) \wedge \text{At}(\text{C2}, \text{JFK}) \wedge \text{At}(\text{P1}, \text{SFO}) \wedge \text{At}(\text{P2}, \text{JFK}) \wedge \text{Cargo}(\text{C1}) \wedge \text{Cargo}(\text{C2}) \wedge$
Plane(P1) $\wedge$ Plane(P2) $\wedge$ Airport(JFK) $\wedge$ Airport(SFO))

Goal($\text{At}(\text{C1}, \text{JFK}) \wedge \text{At}(\text{C2}, \text{SFO})$)

Action(Load(c, p, a),

PRECOND: $\text{At}(\text{c}, \text{a}) \wedge \text{At}(\text{p}, \text{a}) \wedge \text{Cargo}(\text{c}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{a})$

EFFECT: $\neg \text{At}(\text{c}, \text{a}) \wedge \text{In}(\text{c}, \text{p})$)

Action(Unload(c, p, a),

PRECOND: $\text{In}(\text{c}, \text{p}) \wedge \text{At}(\text{p}, \text{a}) \wedge \text{Cargo}(\text{c}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{a})$

EFFECT: $\text{At}(\text{c}, \text{a}) \wedge \neg \text{In}(\text{c}, \text{p})$)

Action(Fly(p, from, to),

PRECOND: $\text{At}(\text{p}, \text{from}) \wedge \text{Plane}(\text{p}) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(\text{p}, \text{from}) \wedge \text{At}(\text{p}, \text{to})$)

PDDL – Limitations

Less expressive than first order logic.

- ▶ No way to say move all cargo to SFO (no universal quantifier in goal).
- ▶ No way to concisely express global constraints: no more than 100 planes at a given airport.
- ▶ No explicit time representation: Cargo must be at JFK by 5PM.

Solving Planning Problem

How do we solve a planning problem represented in PDDL?

- ▶ State Space Search
 - Progression Search
 - Regression Search
 - Heuristics

Progression (Forward) State Space Search

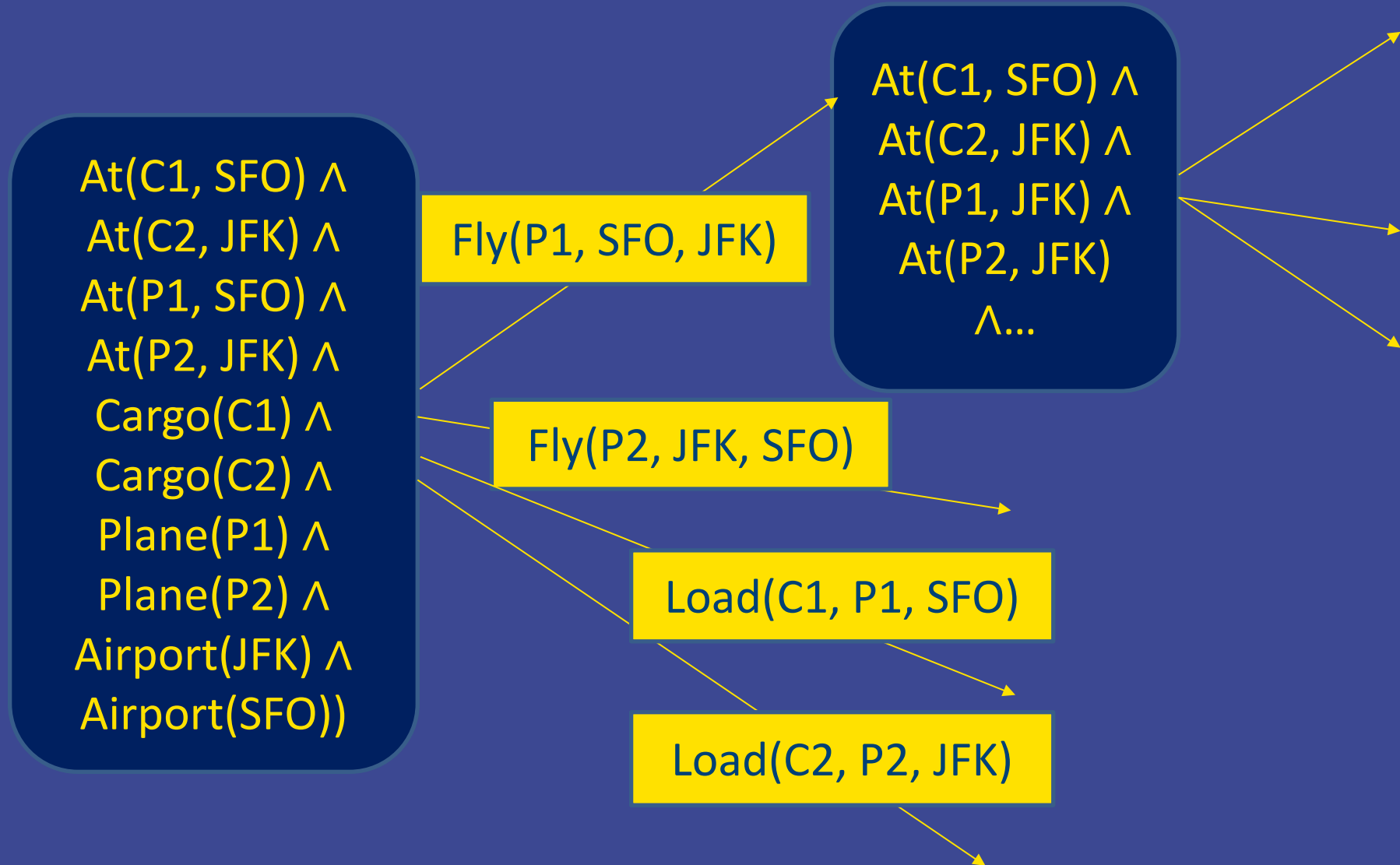
Start with the initial state and apply each possible action repeatedly hoping to find a goal state.

- Unify elements of state with preconditions to find applicable actions.
- Use effects to construct next state: $(s - \text{Del}(a)) \cup \text{Add}(a)$

Reminder: Delete and Add lists of an action a :

- $\text{Del}(a)$: the negative literals in the action's effect
- $\text{Add}(a)$: The positive literals in the action's effects.

Progression (Forward) State Space Search



Progression State Space Search

Problem with progression search?

- ▶ exploring **irrelevant actions**
- ▶ if our goal is to get cargo to JFK, we should not fly any planes before loading the cargo.

Progression State Space Search



Progression Search – Irrelevant Actions

Goal(Own(0134610997))

Init(ISBN(0134610997), ISBN(0134610998), ISBN(0134610999),...)

Action(Buy(i),

PRECOND: ISBN(i)

EFFECT: Own(i))

Our goal is to own the ALMA textbook.

How many applicable actions?

A. 0

B. 1

C. 2

D. 3

E. Billions

Progression Search – Irrelevant Actions

Goal(Own(0134610997))

Init(ISBN(0134610997), ISBN(0134610998), ISBN(0134610999),...)

Action(Buy(i),

 PRECOND: ISBN(i)

 EFFECT: Own(i))

Our goal is to own the ALMA textbook.

ISBNs are 10 or 13 digits, so this action schema has at least 10 billion ground actions (that's a huge branching factor).

An uninformed forward search will enumerate them all.

Progression State Space Search Feasibility?

Only with strong heuristics.

PDDL representation makes it easy to automatically generate heuristics.

Regression Search

Start from the goal, and search backward until we reach a state implied by the initial state. Note that the goal does not represent a single state, but rather a family of states.

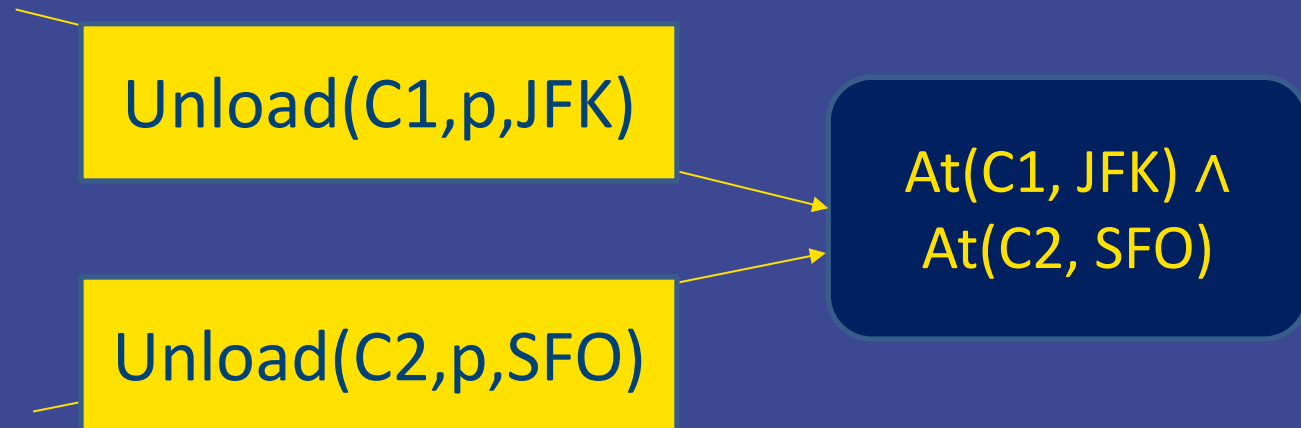
- Unify elements of state with effects to find relevant actions.
- Use preconditions to construct previous state:
 $(s - \text{Add}(a)) \cup \text{Precond}(a)$

Regression Search

$\text{At}(\text{C1}, \text{JFK}) \wedge$
 $\text{At}(\text{C2}, \text{SFO})$

For an action to be relevant, one of its effects must unify with an element of the goal.

Regression Search

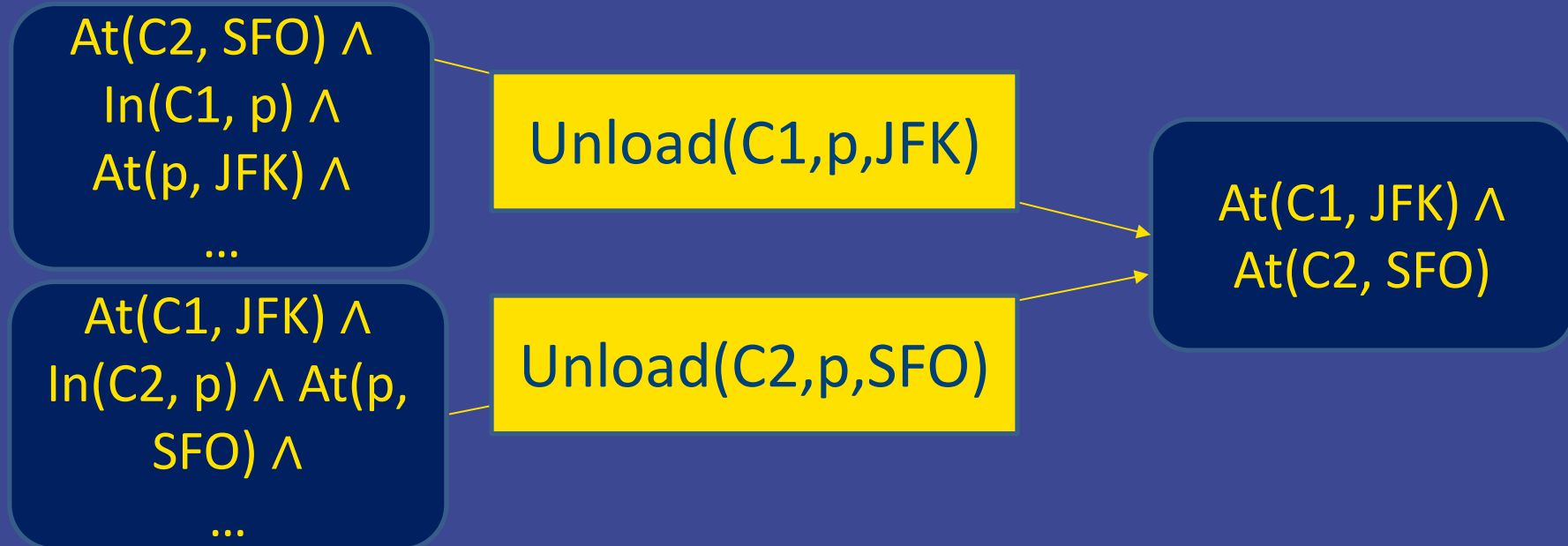


Action(Unload(c, p, a),

PRECOND: $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\text{At}(c, a) \wedge \neg \text{In}(c, p)$)

Regression Search

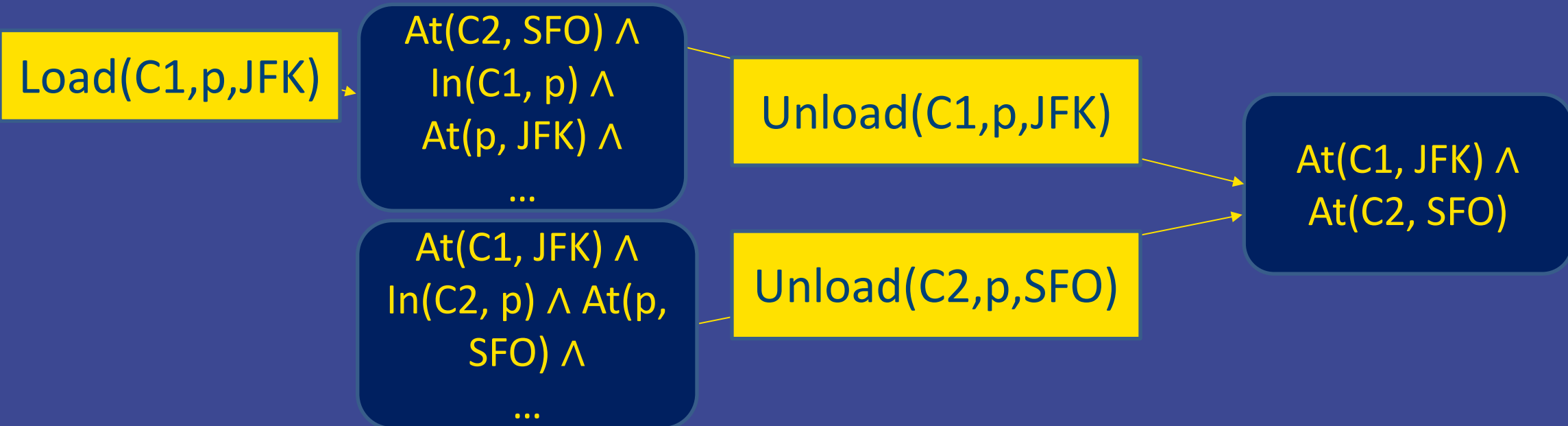


Action(Unload(c, p, a),

PRECOND: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

EFFECT: $At(c, a) \wedge \neg In(c, p)$)

Regression Search

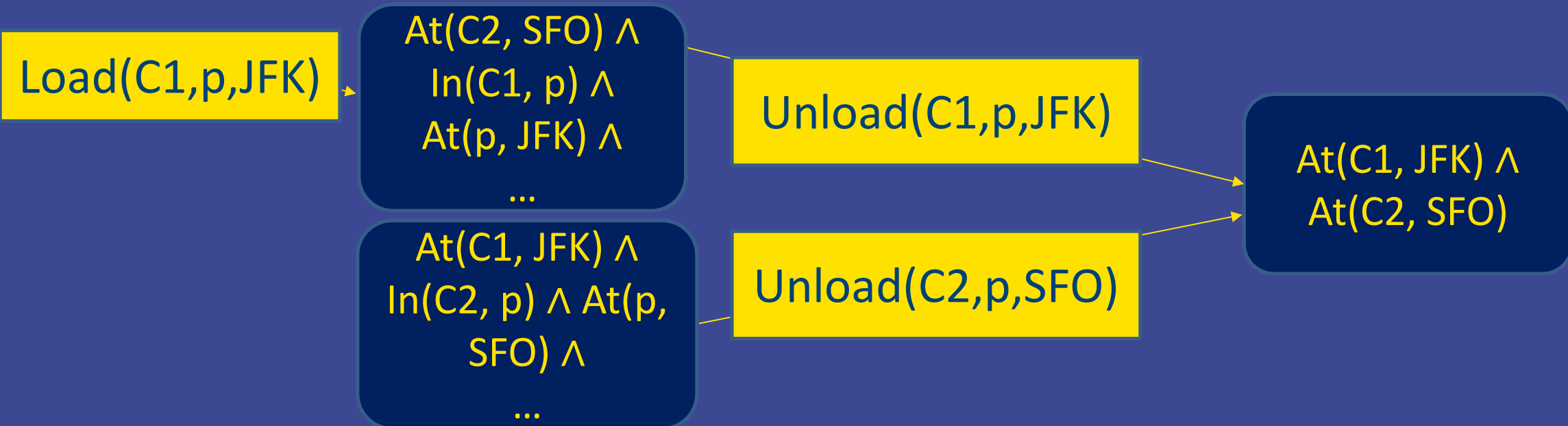


Action(Load(c, p, a),

PRECOND: $At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

EFFECT: $\neg At(c, a) \wedge In(c, p)$

Regression Search

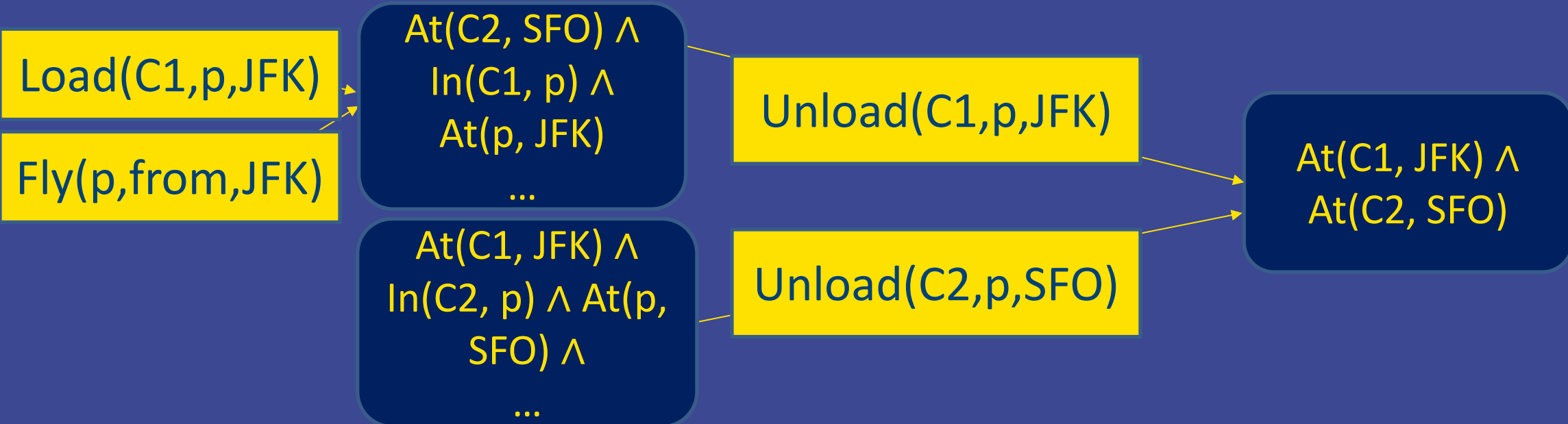


Action(Fly(p, from, to),

PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

Regression Search



Action(Fly(p, from, to),

PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

Regression Search

Goal(Own(0134610997))

Init(ISBN(0134610997), ISBN(0134610998), ISBN(0134610999),...)

Action(Buy(i),

PRECOND: ISBN(i)

EFFECT: Own(i))



For an action to be relevant, one of its effects must unify with an element of the goal.

Heuristics

PDDL representation makes it easy to automatically generate heuristics.

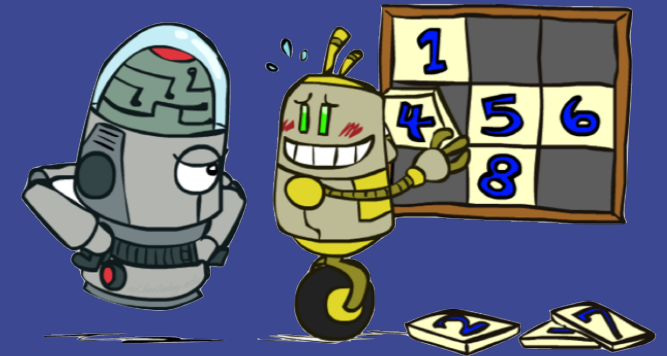
Heuristics are exact solutions to a relaxed problem.

We relax the problem by ignoring preconditions.

Heuristics – Sliding Puzzle

Action(Slide(t, s1, s2),
PRECOND: ?

Predicates:
On(tile, square)
Blank(square)
Adjacent(s1, s2)



Heuristics – Sliding Puzzle

Action(Slide(t , $s1$, $s2$),

PRECOND: ?

Predicates:

On(tile, square)

Blank(square)

Adjacent($s1$, $s2$)

iClicker:

- A. PRECOND: $\text{On}(t, s1) \wedge \neg \text{On}(t, s2)$
- B. PRECOND: $\text{On}(t, s1) \wedge \neg \text{On}(t, s2) \wedge \text{Adjacent}(s1, s2)$
- C. PRECOND: $\text{On}(t, s1) \wedge \text{Blank}(s2)$
- D. PRECOND: $\text{On}(t, s1) \wedge \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$
- E. None of the above



Heuristics – Sliding Puzzle

Action(Slide(t , $s1$, $s2$),

PRECOND: $\text{On}(t, s1) \wedge \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$

EFFECT: ?

Predicates:

$\text{On}(\text{tile}, \text{square})$

$\text{Blank}(\text{square})$

$\text{Adjacent}(s1, s2)$

iClicker:

A. $\text{On}(t, s2) \wedge \text{Blank}(s1)$

B. $\text{On}(t, s2) \wedge \neg \text{On}(t, s1) \wedge \text{Adjacent}(s1, s2)$

C. $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \neg \text{On}(t, s1) \wedge \neg \text{Blank}(s2)$

D. $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \text{Adjacent}(s1, s2)$

E. $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \neg \text{On}(t, s1) \wedge \neg \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$

Heuristics – Sliding Puzzle

Action(Slide(t , $s1$, $s2$),

PRECOND: $\text{On}(t, s1) \wedge \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$

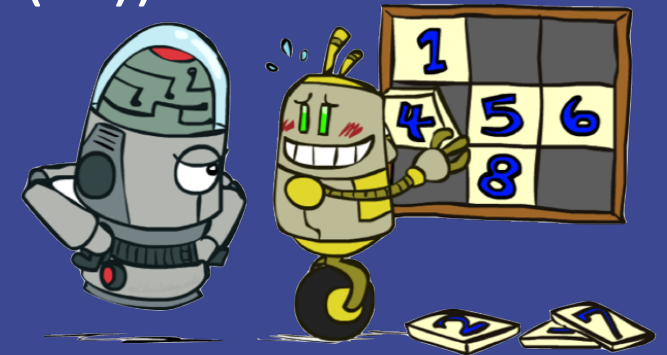
EFFECT: $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \neg \text{On}(t, s1) \wedge \neg \text{Blank}(s2)$)

Predicates:

$\text{On}(\text{tile}, \text{square})$

$\text{Blank}(\text{square})$

$\text{Adjacent}(s1, s2)$



Heuristics – Ignore Preconditions

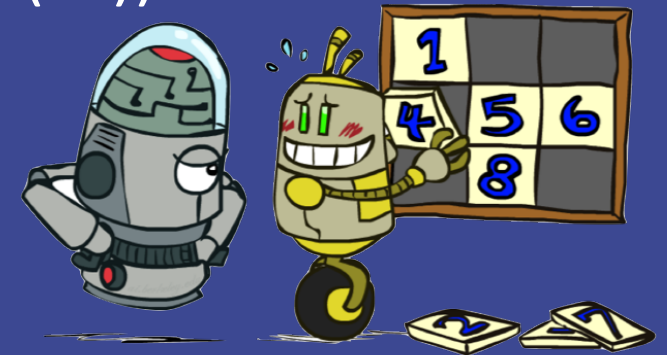
Action(Slide(t, s1, s2),

PRECOND: ~~$\text{On}(t, s1) \wedge \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$~~

EFFECT: $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \neg \text{On}(t, s1) \wedge \neg \text{Blank}(s2)$)

Remove preconditions from action schemas:

- All actions become applicable
 - Every element in the goal is achievable in one step
 - Number of steps for the solution is the number of unsatisfied goal elements?
- ⇒ Heuristic: Number of tiles misplaced

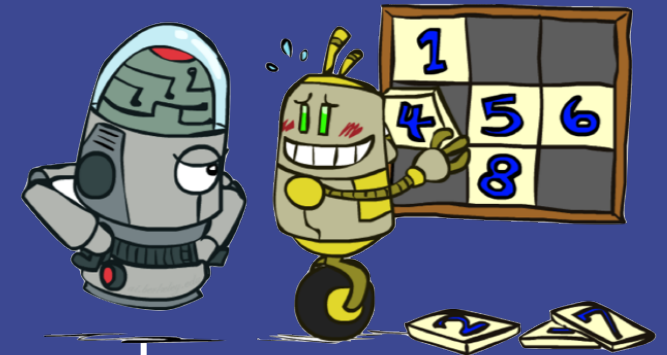


Heuristics – Ignore Preconditions

Action(Slide(t , $s1$, $s2$),

PRECOND: $\text{On}(t, s1) \wedge \text{Blank}(s2) \wedge \text{Adjacent}(s1, s2)$

EFFECT: $\text{On}(t, s2) \wedge \text{Blank}(s1) \wedge \neg \text{On}(t, s1) \wedge \neg \text{Blank}(s2)$)



We can also remove **selected** preconditions from action schema

⇒ Heuristic: Total Manhattan distance to target position

Next

- ▶ QoW on Tuesday
- ▶ Homework 5 due Tuesday October 28
- ▶ Coming up: The tools that got us out of the AI winter
 - Uncertainty
 - Learning